Particle prefab settings guide

ELEC 328 / Assignment 3 / Unity 6.6 + Universal Render Pipeline

Use your existing, working Reality Roost project. The package adds **Fountain, Sparks, and Crowd**, one shared script, meshes, materials, and this guide. Your Roost rig, scenes, and project settings stay under your control.

1. Import and place one prefab

1. In your project, choose **Assets > Import Package > Custom Package** and select `ELEC328_Assignment3_Particle_Prefabs.unitypackage`. Import the listed assets.
2. Open `Assets/ELEC328ParticlePrefabs/Prefabs`. Drag one prefab into your experience scene. Keep it outside the rails and leave its scale at `(1,1,1)`. If your rails are centered at the origin, `(0,0,12)` is a starting position; check the full extent of the moving particles.
3. Enter Play mode using your usual Roost workflow. Select the prefab instance in the Hierarchy and use its **Particle Benchmark** component. Disable other particle instances for controlled comparisons.

2. Change settings in the Inspector

Setting	What to change or observe
Render Strategy	Switch GameObjects / Instanced. Both use the same CPU simulation.
Particle Count	Try small, medium, and large counts, for example 100 / 1,000 / 5,000.
Animate	Uncheck to pause the simulation; rendering continues.
Simulation Speed	Change the motion rate; keep population and rendering fixed.
Random Seed	Sets the reset pattern. After changing it, use Reset Simulation.
Automatic Spark Bursts	Sparks only: check for repeated bursts; uncheck for manual triggering.
Spark Burst Interval	Sparks only: time between automatic bursts, in simulation seconds.

Leave **Visual** and the **Shared Assets** unchanged within a controlled comparison. The component's three-dot menu includes **Reset Simulation**, **Pause or Resume**, and **Trigger Spark Burst**. Use these during Play mode. Changing count reseeds particles; switching rendering mode preserves their state.

3. Connect a simple friend interaction

On an existing XRI interactable, add a **Select Entered** event. Drag the particle prefab instance into the event's object field, then choose `ParticleBenchmark.TogglePause()` or `ParticleBenchmark.TriggerSparkBurst()`. For manual sparks, uncheck **Automatic Spark Bursts**. The same methods can be called by your own interaction script.

Fountain is a continuous droplet system. Crowd is a bobbing capsule field. Sparks has a changing active population: Particle Count is its burst capacity. Use Fountain or Crowd for a steady-population baseline.

Compare settings and gather evidence

Use the same computer, view, resolution, and profiling setup for each trial

4. Change MSAA in your existing URP asset

1. Open **Edit > Project Settings > Quality** and identify the render-pipeline asset for the active quality level. If there is no quality override, use the default asset in **Project Settings > Graphics**. Select that asset in the Project window.
2. In the URP asset Inspector, find **Quality > Anti Aliasing (MSAA)**. Compare supported settings: **Disabled, 2x, 4x, 8x**. Keep render scale, population, dynamics, and viewpoint fixed.
3. Use a compatible **Forward or Forward+** renderer; Deferred does not support MSAA. Ensure the viewing camera allows MSAA. Keep camera post-process anti-aliasing off to isolate the comparison. Check actual edge quality during motion; a requested sample count may be limited by the device.

Prefab Inspector edits in Play mode normally revert when you stop. Changes to the URP asset are asset edits and persist. Record the original MSAA setting and restore it afterward.

5. Open Profiler and Stats

Open **Window > Analysis > Profiler**. Record the **CPU Usage** module and inspect **Timeline** or **Hierarchy**. Search for the supplied markers:

```
Assn3/Particle Simulation
Assn3/GameObject Transform Updates
Assn3/Instanced Draw Submission
```

In the **Game view**, enable **Stats**. Record the exact counter labels you see. **Batches/draw calls** describe rendering submissions; **SetPass calls** count shader-pass switches. They are different measures. Use measured counters rather than estimating them from population.

6. Run the comparison with a friend

1. Have your friend repeat the same interaction from a similar viewing position while you change one setting at a time. Compare both rendering modes at three counts; then vary dynamics and MSAA as directed by the assignment.
2. After each change, allow at least **5 seconds to settle**, then observe at least **5 seconds**. Use matching intervals, note spikes, and repeat inconsistent trials. Avoid measuring the initial GameObject creation/destruction stall.
3. For each trial, record the settings, typical **CPU frame time (ms)**, observed range/spikes, **Batches or draw calls**, **SetPass calls**, and your friend's observations. Identify hardware, resolution, VSync/frame cap, and whether results came from Editor Play mode or a player. Mark unavailable GPU timing as **N/A**.

Measure the machine that renders the experience. In a networked Roost session, the host Editor's Profiler does not measure a remote headset. These prefabs add no networking: Inspector changes on one computer are local. For the simple exercise, use a headset driven by the same Editor, or configure each rendering client separately.

Unity references: URP asset and MSAA / Rendering Stats / CPU Usage Profiler. Follow the released assignment for required trials and submission details.